SmolVLA on the KR260: the measured path from 24 s to ~3 s per chunk, and to continuous motion

For psamin, from the FPGA side (athithan-elamaran1). 2026-10-22. Everything marked **measured** was run today: the real checkpoint (`smolvla-so101_medicine_bottle_pickplace-9604`, release weights, SHA verified) on the 3 recorded frames, 49 arithmetic variants (147 chunk evaluations); 57 Vitis HLS syntheses on the K26 part; the board runs in `docs/smolvla-fpga.md`. Everything else is an estimate from a cost model calibrated on those runs (it reproduces today's 24.4 s to 0.1 s) and is labelled so. Scripts and raw outputs: `build/brief/` on the FPGA laptop.

Summary

Where we are. 24.4 s per 50-action chunk on the board with the model exactly as designed (BF16 weights, FP32 math), max |chunk - PyTorch FP32| = 0.0043 normalized against the 0.05 gate. Engine calls 12.5 s (FPGA busy ~9 s at 128 FP32 MACs/clock; copies and call overhead ~3.5 s); ARM ~12 s (vision attention 7.5 s, GELU 2.2 s, copies/norms/Python ~2.5 s). **FP32 is at its ceiling on this chip**: a double-width (TP = 8) FP32 engine was built and needs 118,727 LUTs of 117,120 (Vivado DRC); the best FP32 recipe we can model is ~12 s.
What the measurements say about precision. (1) **int16 is the robust choice**: int16 weights (per-output-row scale) x int16 activations (per-token scale) with exact integer accumulation gives **0.00034** on the worst frame and 0.00014 on the other two - 12x and 10x *better* than today's exact-FP32 board - and an int16 MAC costs **1 DSP + 24-60 LUTs** (synthesized; 24 with the accumulate inside the DSP, 60 with fabric accumulators) against 5 DSPs + ~480 LUTs for FP32, so a 1,024-MACs/clock engine fits at 60-80 % of the LUTs. (2) **int8 is not needed and is the risky one**: symmetric per-token int8 activations land at 0.56 on the sensitive frame (0.03 on the benign ones) because a few layers (LM down-projection, connector, vision fc2) carry activation outliers; an asymmetric (zero-point) quantizer brings that to 0.030 / 0.017 / 0.021 (gate edge on the sensitive frame, ~1.5x Option A on the benign ones), and on this chip int8 is not faster than int16 anyway (both DSP-bound; the int8 engine needs 3x the LUTs for 1.25x the MACs at a lower clock). (3) **'BF16 products with FP32 accumulate' (Q13) buys nothing here**: the FP32 adder is the cost, not the multiplier; synthesized BF16 MAC variants came out at 600-960 LUTs, worse than FP32. BF16 activations themselves are harmless (0.0037). (4) **A free accuracy win**: today's 0.0043 is not the FPGA's arithmetic but the BF16 rounding of 21 small weight tensors the checkpoint stores in FP32 (3.7 of 255 GMAC); computed in FP32 on the ARM, the board matches PyTorch to 3e-6. (5) **The acceptance test needs rethinking**: frame 0 is a motion-onset frame (the reach starts at step 45 of 50) and in exact FP32 one 8-bit level of camera noise moves its chunk by 0.11-0.16, far over the 0.05 gate; the other two frames are nearly static. The 3-frame gate cannot rank precision options finer than ~0.05; the arm success rate can.
What 3 s per chunk does and does not give. With LeRobot's async client the arm executes the *second half* of each chunk and discards the half that is already in the past, so at 3 s per chunk it moves 0.83 s in every 3 s (duty 28 %; today 3 %). Continuous motion needs the chunk to arrive within 0.82 s of its observation (1.1 s with two inferences in flight) because a chunk only covers 1.67 s - independent of how fast the FPGA gets, unless the model produces longer chunks.
Recommendation. Tier 1 (about a week): int16 engine + exact software levers (FPGA side) and the vision tower at 256 px (your re-finetune of the expert: 2.4x fewer linear MACs, 16x less attention) -> **~4-5 s per chunk, ~1 s of motion per 4 s**, precision better than today; the int16 engine alone, at 512 px, lands at ~14-16 s because the ARM's attention then dominates. Tier 2 (3-5 weeks more): attention/softmax/GELU/norm kernels on the FPGA + a command sequencer + two to three inferences in flight -> **~1.2-1.5 s per chunk, continuous** (the full int16 recipe at 512 px is ~3.5 s, 2.5-2.8 s with the sequencer). **Decisions needed from you**: (a) int16 arithmetic acceptable (measured 0.00034 vs today's 0.0043; 0.00014 vs 0.0013 on the benign frames)? (b) a 256-px re-finetune of the expert this week (the live client already downsizes frames to 256 px before the policy upsamples them to 512, so the tower sees 256-px information today)? (c) the acceptance test beyond the 3 frames: an arm success-rate you consider 'the same policy'.

1. Where the time goes and what the model needs per chunk (measured)

Today's 24.4 s	s	Work per chunk (frame 0, hooked)	amount
FPGA busy (128 FP32 MACs/clock @ 250 MHz)	~9.0	nn.Linear calls	1,419 = vision 144 + connector 2 + LM 112 + expert 1,120 + proj 41
copies into/out of CMA + flush/invalidate (2.05 GB)	~3.0	linear MACs	255.7 G = vision 174.0 + LM 27.8 + expert 51.6 + connector 1.5 + proj 0.8
engine call overhead (registers, poll, Python)	~0.5	attention MACs	42.7 G = vision 38.65 (24 calls, 12 heads x 1024 x 1024 x 64) + LM/expert 4.1
ARM vision attention (torch flash kernel, ~5 GMAC/s)	7.5	softmax exps / tanh-GELU / SiLU	320 M / 75.5 M / 23.6 M elements
ARM tanh-GELU (sigmoid form, 34 M elem/s)	2.2	norms	LayerNorm 39 M + RMSNorm 17 M elements
ARM norms, residuals, RoPE, flow matching, Python	2.3	activations through the engine / weights streamed	1.01 GB in + 1.04 GB out / 17.0 GB (each layer re-read per 32-token tile)

Prefix = 177 tokens: 2 x 64 image tokens (1,024 patches per 512-px image, pixel-shuffled 4x4) + 48 language (12 real + 36 padding) + 1 state; the image + language block is fully bidirectional, so nothing of the prefix can be cached across chunks; 10 Euler steps x 16 expert layers on 50 action tokens (prefix K/V cached across steps).

2. Precision options, measured on your checkpoint (3 frames; error = max |chunk - PyTorch FP32 reference|, normalized; gate 0.05)

Variant (what is rounded where)	frame 0	f3563	f7489	worst	x today	gate
FP32 everything (laptop control): the reference regime	3.6e-7	4.2e-7	5.4e-7	5.4e-7	0	pass
V0 today's board: weights BF16 (as stored), FP32 math, the engine's own summation order	0.0043	0.0013	0.0011	0.0043	1.0	pass
V0 + the 21 FP32-stored layers (policy projections, expert cross k/v; 3.7 GMAC) kept in FP32	3.0e-6	6.4e-7	5.4e-7	**3.0e-6**	0.001	pass
V1 BF16 activations into every linear, FP32 accumulate (your Q13 'Option A', linears)	0.0036	0.0037	0.0033	0.0037	0.9	pass

Variant (what is rounded where)	frame 0	f3563	f7489	worst	x today	gate
V2 = V1 + Q, K, V, P in BF16 in every attention, FP32 softmax/accumulate	0.0257	0.0039	0.0033	0.0257	6.0	pass
A_full = V2 + FP16 softmax/GELU inputs (cheap exp unit); with the 21 FP32 layers kept FP32	0.0304 / 0.0127			0.0304 / 0.0127	7 / 3	pass
V4 int16 weights (per-row) x int16 activations (per-token), exact integer sums	3.4e-4	1.4e-4	1.0e-4	**3.4e-4**	**0.08**	**pass**
V5 int8 weights x int16 activations	0.1213	0.0049	0.0050	0.1213	28	fails frame 0
V3 int8 x int8, symmetric per-channel / per-token, exact int32 sums (all 303 linears)	0.5625	0.0290	0.0303	0.5625	131	fails frame 0
int8 weights x **asymmetric** (zero-point) per-token uint8 activations, exact sums (reviewer run)	0.0304	0.0171	0.0211	0.0304	7	pass (edge)
int8 weights only (RTN or MSE-clipped), FP32 / BF16 / int16 activations	-	0.0050	0.0050	benign 0.0050	-	the activations are the problem
symmetric int8 only in SigLIP (68 % of MACs) / only connector / only LM / only expert	0.278 / 0.037 / 0.390 / 0.079					fails frame 0 (except connector)
symmetric int8 best efforts: SmoothQuant + group-64 scales + BF16 expert, int8 attention (M2_full / V6s)	0.128 / 0.178	0.011 / 0.017	0.017 / 0.019	0.128 / 0.178	30 / 41	fails frame 0
control: exact FP32 with one 8-bit level of Gaussian camera noise (sigma 1/255), 3 seeds	0.111 / 0.122 / 0.162	0.0015-0.0033		0.162	38	sensor-noise response
V7 FP16 inputs to softmax and GELU only (cheaper transcendental unit)	0.0026	0.0013	0.0010	0.0026	0.6	pass
V9 whole policy in torch bf16 autocast (the GPU bf16 regime, BF16 outputs)	0.0283	0.0040	0.0046	0.0283	6.6	pass
V8 fewer Euler steps, FP32: 10 -> 5 / 10 -> 3 (sampler change, judged by task success)	0.114 / 0.167	0.023 / 0.038	0.021 / 0.048	0.114 / 0.167	26 / 39	model change

Reading. The three frames are not equal: frame 0 is episode 0 at the rest pose and its chunk contains the *onset* of the reach at step 45 of 50 (elbow range ~30 deg), so every variant's frame-0 error is the timing of that onset in the last five steps - exact FP32 with one 8-bit level of camera noise moves it by 0.11-0.16, and 0.01 of state noise by 0.105; frames 3563 and 7489 are end-of-episode chunks with 1-3 deg of motion. Ranking on the benign frames (max of the two): today's board 0.0013; int16 0.00014; BF16 activations (Option A) 0.0037, + BF16 attention 0.0039, A_full 0.0040, torch bf16 autocast 0.0046; int8 weights only 0.0050; asymmetric int8 activations in SigLIP + BF16 elsewhere 0.0049, in the expert 0.0053; symmetric int8 0.030. On frame 0 the same options spread from 0.003 to 0.56, and BF16-level rounding with five random rounding seeds alone spans 0.006-0.061 there, so frame 0 cannot rank them. What is robust: int16 is two orders of magnitude below every other option on every frame; symmetric int8 is unsafe on motion onsets because of activation outliers (per-token absmax/rms of 10-16 in the connector, LM down with absmax up to 295 vs typical 1.7, vision fc2 and q/k/v leave ~3 bits for a typical channel); asymmetric int8 recovers most of it. Method: fake quantization in torch (operands rounded, exact integer or FP32 accumulation), dynamic per-token / per-channel scales, round-to-nearest-even; the reviewer re-implemented the quantizers independently and reproduced V0, V1, V3, V4 to all printed digits. Full table (49 rows + the review's runs): build/brief/precision_results.md, build/brief/review_e/.

3. Hardware: what each arithmetic costs on the K26 (synthesized) and what fits

MAC arithmetic (Vitis HLS 2022.2, xck26, II = 1)	LUT/MAC	FF/MAC	DSP/MAC	post-route LUT*	note
FP32 x FP32 -> FP32 (today: fmul maxdsp + fadd full_dsp)	482	904	5.0	480 all-in**	reproduces the shipped engine's compute block (64.4k LUT at TP = 4)
BF16 x BF16 exact product -> FP32 accumulate	598-639	~710	2-3	419-447	worse than FP32: the fadd (236 LUT + 2 DSP) is the cost
BF16 products -> 48-bit integer accumulator with running exponent	962	594	0	674	two barrel shifters per MAC
BF16 W (per-row exponent) x int24 block-float X -> int48	305	112	1.0	214	exact BF16 weights, 24-bit activations per token
int16 x int16 (or int24 x int16) -> int48	**24 / 60**	48 / 96	1.0	~24-60	24 with the accumulate inside the DSP48E2 (one token block per tile), 60 with fabric accumulators (several blocks); reset outside the loop
int8 x int8 -> int32, two MACs per DSP (packed)	66	41	0.5	47	fabric accumulators; barely faster than int16 on this part

* HLS estimates; Vivado landed at 0.68-0.74x of them for the FP32 core logic, but ** the measured all-in slope of the real FP32 engine (TP 4 -> 8, post-route) is 480 LUT per MAC, and no post-route data exists for integer fabric logic, so the integer rows are used at 1.0x below.

Largest engine that fits (85 % LUT/FF, 95 % DSP headroom, 12k LUT infrastructure)	MACs/clock	LUT	DSP	clock plan	255 GMAC ideal
FP32 (today TP = 4 is 128; TP = 8 measured not to fit: 117.5k LUT)	160-192 (TP 5-6)	61-74 %	65-78 %	225 MHz	6-7 s
BF16 products, FP32 accumulate	192	75 %	47 %	200 MHz	6.7 s
BF16 W x int24 X, int48 sums	384	77 %	32 %	200 MHz	3.3 s
int16 W x int16 X, int48 sums (fabric accumulators, 2 token blocks per tile)	**1,024**	**60-83 %**	83-94 %	200-225 MHz	**1.1-1.3 s** (weight stream becomes the bound at small tiles)
int8 (packed + unpacked DSP mix)	1,280	74 %	~90 %	200 MHz	1.0 s (1,536 does not fit once the accumulators are counted)

Kernels that take the ARM out of the loop (all synthesized at II = 1, 4 lanes unless noted; per-chunk times at the integer recipe's clock): Q K^T and P V through the engine with the heads of a layer batched per call (0.15 s int16; needs the engine's X load and emit

overlapped and a 16-wide emit, which today's engine lacks - attention-shaped calls are emit-bound); row softmax with fixed-point 2^x tables, 16 lanes, 8.1k LUT + 16 DSP + 16 BRAM, 0.12-0.18 s for 320 M exps; tanh-GELU table (2.4e-5 max error), 4 lanes, 6.7k LUT, 0.07 s; LayerNorm/RMSNorm/SiLU, 4 lanes, ~6k LUT + 95 DSP, 0.2 s; per-token X scale pass, 8 lanes, 0.14 s; patch-embedding conv as an im2col GEMM (5 ms). Engine + kernels use ~75-85 % of the LUTs in the int16 recipe and ~116 of 144 BRAM tiles; the FP32 recipes cannot afford the kernels without dropping engine width. Weight stream: 17 GB per chunk today at 32 tokens per tile; the int16 engine must use 64 tokens per tile (8.5 GB at 2 B/weight; the ~14 GB/s DDR figure is from the literature, not yet measured on this board, and the integer recipes sit within 15-20 % of that floor). The on-chip X tile costs one URAM per token row (no width packing: today 32 URAMs for 32 tokens), so a 64-token tile is 64 URAMs = the whole chip unless the values are packed two per word explicitly (32, double-buffered 64) - the first detail to settle in the integer engine.

Recipe (model: engine rework + driver tightened; no FPGA/ARM overlap assumed)	FPGA s	ARM s	est. s per chunk	precision class	effort
R0 today (measured 24.4)	9.1	15.5	24.5	exact FP32	-
R-A: FP32 engine at TP 4-5 + attention/softmax/GELU/norms on FPGA, K/V BF16	11.5	1.4	12.9 (11.7 @250 MHz)	linears exact; attention operands BF16	6-8 wk
R-A2: BF16 W x int24 X, int48 sums (~320 MACs/clock) + kernels	~6	1.4	~7.5	W exact; X 24-bit per token; exact sums	8-10 wk
R-B16: int16 engine (1,024 MACs/clock, 200-225 MHz) + int attention + fixed softmax + kernels	~2.1	1.4	**3.4-3.6 (2.5-2.8 with a command sequencer)**	measured 0.00034 (V4)	8-12 wk full; **engine alone ~1 wk**
R-B: int8 engine (1,280 MACs/clock) + int8 attention + kernels	~2.0	1.4	>= 3.4	asym int8 at the gate edge on frame 0 (section 2)	10-14 wk

int8 is not faster than int16 on this part (both DSP-bound; packing buys 1.25x MACs for a clock step and 3x the LUTs) and carries the quantization risk: there is no case for it here. Model caveats: the recipes assume a tightened driver (0.1 ms per call), an overlapped X load and a 16-wide emit, no FPGA/ARM overlap, and an ARM glue of 1.2-1.4 s that is not yet measured; the DDR bandwidth and the clock at 90 % DSP use are assumptions until a build exists.

4. Model-side and system-side levers (measured where marked)

Lever	exact or model change	measured error	effect today (24.4 s)	effect on a ~4 s FPGA-dominated budget	effort / owner
fuse q/k/v and gate/up into one engine call each (1,419 -> 1,003 calls); hoist the expert cross-attn K/V projections, identical across the 10 steps (-> 859 calls, -2.6 GMAC); drop the 36 language pad tokens (tokenizer padding 'longest')	exact (bit-identical per output lane; 5.1e-6 for the pad trim)	0 / 0 / 5.1e-6	~1.5 s	~0.3 s	days / FPGA side (fusion exists unreviewed on the wip branch)
batch the two images in one vision pass	exact (bit-identical)	2.1e-6	negative today (loses the image/engine overlap)	<= 0.04 s	hours
vision at 256 px (1,024 -> 256 tokens per image, 16 per image into the LM; prefix 177 -> 81)	**model change: re-finetune the expert** (1.92 normalized = 73 deg without it; the expert was trained on a 177-token prefix)	needs validation	**~3x** (linears 255.7 -> 107 GMAC, attention 38.7 -> 2.4 GMAC)	**~2.3x**	days on the cluster (~10k steps as train_config) / psamin
vision at 384 px (576 tokens)	model change: re-finetune	0.73 without	~1.8x	~1.5x	psamin
num_steps 10 -> 5 / 3	model change (sampler)	0.114 / 0.167 on frame 0 (3.3 / 4.9 deg)	1.11x / 1.16x	1.14x / 1.21x	hours / psamin judges by task success
one camera	model change: re-finetune, likely worse success	1.9-2.0 (53-63 deg)	1.7x	1.6x	psamin
longer action chunk (>= 181 actions) for continuous motion at 3 s latency	model change (6 s horizon, 3.6x expert work)	not run	-	-	psamin; base model is 50
fix the live pipeline's 256-px squash: the client resizes camera frames to the config's (3, 256, 256) before the policy's resize_with_pad(512), so the tower sees a 2x-upsampled, aspect-squashed image (mean pixel diff 0.22-0.34 vs the recorded frames)	restores training-time inputs; changes live behaviour	-	0 s	0 s	hours / both

5. The control loop: what 'continuous' requires (LeRobot async client, simulated from its rules)

chunk latency	inferences in flight	duty (fraction of time moving)	actions executed per chunk	age of executed actions
0.5 s	1	100 %	34	0.5-1.3 s
0.8 s	1	100 %	25	0.8-1.6 s
1.0 s	1 / 2	81 % / 100 %	25 / 19	1.0-1.9 s
1.5 s	1	54 %	25	1.5-2.4 s
3.0 s	1 / 2 / 3	**28 % / 36 % / 41 %**	25 / 17 / 12	3.0-3.9 s
4.0 s	1	20 %	25	4.0-4.9 s
24.4 s (today)	1	3 %	25	24-25 s

Rules (robot_client.py / policy_server.py): an observation is sent when the queue is at or below the threshold (0.5 x 50 = 25 actions left); chunk action i carries timestep observation + i; actions with a timestep already executed are dropped, overlapping ones blended 0.3 old / 0.7 new; the arm holds when the queue is empty. Hence: continuous motion iff latency <= 0.82 s with one inference in flight

(<= 1.1 s with two, 1.2 s with three); raising the threshold changes nothing; a plain synchronous loop gives duty 1.67 / (1.67 + latency) = 36 % at 3 s. LeRobot's RTC needs latency < the 1.67 s chunk as well. So **3 s per chunk means ~1 s of motion every 3 s**; continuous needs <= ~1 s per chunk or a longer action horizon.

6. The plan

Tier	Content	Expected (est.)	Precision	Time
1. int16 engine (FPGA side)	HLS: int16 weights per-row scale, int24 activations per-token scale, int48 accumulate in the DSP, dequantize in the emit path, same top-level ports (the Vivado install cannot regenerate IPs, so the IP is upgraded in place); runtime: weight requantization, per-token scaling on the ARM, fused q/k/v and gate/up calls, copy cuts; one Vivado build; board selftest on the 3 frames	~14-16 s per chunk at 512 px (linears ~1.3 s; the ARM's attention, GELU and copies now dominate)	measured 0.00034 on the frames (12x better than today); FP32 for the 21 FP32-stored layers	3-5 days
2. 256-px re-finetune (psamin)	retrain the expert with resize_imgs_with_padding = (256, 256) (frozen tower/LM unchanged; position embedding is bucketized so the 16x16 grid is valid), validate on the frames and the arm	with tier 1: **~4-5 s** (linears ~0.6 s, ARM ~3-4 s)	to be measured	2-3 days, parallel
3. kernels + sequencer (FPGA side)	attention through the engine with heads batched, fixed-point softmax, GELU table, norm/SiLU kernels, X scale pass, overlapped X load / 16-wide emit, command list so the ARM issues one list per layer	~1.2-1.5 s per chunk (R-B16 at 256 px with the sequencer)	softmax tables 2e-4 relative; GELU table 2.4e-5; to be measured end to end	2-4 weeks
4. client	two to three inferences in flight (activation sets in CMA), threshold tuned	**continuous** (100 % duty at <= 1.1-1.2 s)	-	1-2 days

One-week outcome: tiers 1 + 2 (+ the exact levers) -> ~4-5 s per chunk, ~1 s of motion per 4 s, duty ~25-35 %, precision inside the gate with margin. **Continuous**: tiers 3 + 4, 3-5 weeks more. **Risks**: timing closure of the int16 engine at 83-94 % DSP use and 60-80 % LUTs (planned 200-225 MHz; every clock step costs ~10 % of the engine time); the X tile against 64 URAMs (needs explicit packing); the first integer engine on this team (the shipped FP32 engine's tests, C model and board chain carry over); the re-finetune's task success at 256 px is unknown until the arm run. **What we need from you**: the three decisions in the summary, the calibration/validation protocol you want applied to the re-finetune, and a yes on int16 before it becomes the deployed policy (building and measuring it is on us).

7. Method and files

Precision: build/brief/precision_emulation.py (torch monkeypatches on the loaded policy; dynamic absmax scales; exact integer accumulation via int32 matmul or float64), results precision_results.{json,md}. Hardware: 57 HLS syntheses of a 32-lane MAC array at TP 2 and 4 (per-MAC cost = slope), non-linear kernels at 4 lanes (build/brief/hls_*/, hardware_options.md), cost model costmodel.py (calibrated to 24.4 s). Model/system: count_calls.py (hooked call/MAC/traffic counts), levers_eval.py (each lever run on the 3 frames), async_sim.py (the client's rules), model_system_levers.md with the code lines. Each report was reviewed by an independent agent that re-ran the key items; their corrections are applied here (the frame-0 motion-onset analysis and the asymmetric-int8 result come from the precision reviewer's independent runs; citation lines, a 14 -> 139 MB traffic figure and register-write counts from the model/system reviewer). Caveats: the 3 frames are the project's acceptance set, not a statistical sample; chunk-time estimates carry the model's uncertainty (the ARM 'glue' of 1.2-1.4 s is the least certain); HLS resource counts are pre-place-and-route (the FP32 engine's measured all-in slope, 480 LUT per MAC, is used for FP32; integer fabric logic at 1.0x); the hardware reviewer's corrections (accumulator structure, URAM packing, DDR floor, clock assumptions) are applied. Shipped state: branch fpga/203-float-engine at 792ffaf, PR #214; unreviewed kernel work (exact attention through the engine, FPGA softmax mode, fusions): branch fpga/wip-kernels-2026-10-22.